

Clase 237 — Modelado de amenazas: STRIDE y DREAD

Parte: 11 — DevSecOps y seguridad del SDLC · Fuente: Threat Modeling: Designing for Security (Adam Shostack) y OWASP Threat Modeling Cheat Sheet 🕒 Duración estimada: 120 min · Nivel: Intermedio

Objetivo

Aprender a modelar amenazas de forma sistemática antes de escribir código: descomponer un sistema en un diagrama de flujo de datos (DFD), identificar amenazas con STRIDE, priorizarlas con un método de riesgo (DREAD u otro), y traducirlas en requisitos y controles concretos. El modelado de amenazas es la práctica shift-left de mayor retorno: previene defectos de diseño que ninguna herramienta automática detecta.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Construir** un diagrama de flujo de datos (DFD) con límites de confianza (trust boundaries).
- 2. **Aplicar** STRIDE a cada elemento del DFD para enumerar amenazas.
- 3. **Priorizar** amenazas con DREAD y reconocer sus limitaciones frente a alternativas (CVSS, ábaco de riesgo).
- 4. **Derivar** contramedidas y requisitos de seguridad a partir de las amenazas.
- 5. **Documentar** el modelo de forma reutilizable y versionable junto al código.

Temas

#	Tema	Por qué importa
1	Las 4 preguntas de Shostack	Marco simple: qué construimos, qué puede salir mal, qué hacemos, revisamos
2	Diagramas de flujo de datos (DFD)	Modelo visual sobre el que razonar amenazas
3	Trust boundaries	Donde cambia el nivel de confianza es donde acechan las amenazas
4	STRIDE	Taxonomía mnemotécnica de 6 categorías de amenaza
5	DREAD y sus críticas	Método de scoring subjetivo; conocer sus límites
6	Threat modeling as code	Versionar el modelo (pytm, Threat Dragon)
7	De amenaza a contramedida	El objetivo real: requisitos accionables

Definiciones y características

- **Modelo de amenazas:** representación estructurada de qué puede atacar a un sistema y cómo mitigarlo. *Característica:* se hace en fase de diseño, es barato y preventivo.
- **DFD:** diagrama con procesos, almacenes de datos, flujos y entidades externas. *Característica:* los cruces de trust boundary son los puntos calientes.
- **STRIDE:** Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege. *Característica:* cada categoría se opone a una propiedad de seguridad (autenticación, integridad, no repudio, confidencialidad, disponibilidad, autorización).
- **DREAD:** Damage, Reproducibility, Exploitability, Affected users, Discoverability. *Característica:* scoring 1–10 por eje; útil pero subjetivo, Microsoft lo abandonó por inconsistencia.
- **Trust boundary:** frontera donde datos o control pasan entre niveles de confianza distintos. *Característica:* internet↔DMZ, usuario↔proceso, contenedor↔host.
- **Contramida (mitigación):** control que reduce la probabilidad o el impacto de una amenaza. *Característica:* se traduce en requisito verificable.

Herramientas y preparación

- **OWASP Threat Dragon** (app de escritorio/web) para dibujar DFD y anotar amenazas STRIDE.
- **pytm** (Python) para modelar amenazas como código y generar el DFD e informe.
- **Microsoft Threat Modeling Tool** (Windows) como alternativa gráfica.
- Plantilla de tabla de amenazas (elemento | STRIDE | descripción | riesgo | mitigación).

Instalación de pytm:

```
pip install pytm
# Requiere Graphviz para renderizar el DFD:
# Debian/Ubuntu: sudo apt install graphviz
# macOS: brew install graphviz
```

Laboratorio guiado

Modelaremos una app web de ejemplo: navegador → API → base de datos, con login.

1. **Define el alcance.** Sistema: API REST de una tienda con autenticación por JWT, base de datos PostgreSQL y un servicio de pagos externo.
2. **Dibuja el DFD.** Entidades externas (usuario, pasarela de pago), procesos (API, servicio de auth), almacén (DB), flujos entre ellos. Marca trust boundaries: internet↔API, API↔DB, API↔pasarela.
3. **Aplica STRIDE por elemento.** Para el flujo "usuario → API login": Spoofing (¿se puede suplantar al usuario?), Tampering (¿se puede alterar el request?), etc. Rellena la tabla.
4. **Modela como código con pytm.** Ejemplo mínimo:

```

from pytm import TM, Server, Datastore, Dataflow, Boundary, Actor

tm = TM("Tienda API")
inet = Boundary("Internet")
dmz = Boundary("DMZ")

user = Actor("Usuario"); user.inBoundary = inet
api = Server("API REST"); api.inBoundary = dmz
db = Datastore("PostgreSQL"); db.inBoundary = dmz

login = Dataflow(user, api, "POST /login (credenciales)")
login.protocol = "HTTPS"; login.isEncrypted = True
query = Dataflow(api, db, "SELECT usuario")

tm.process() # genera hallazgos STRIDE automáticos

```

Ejecuta `python tm.py --report y --dfd | dot -Tpng -o dfd.png`. 5. **Prioriza**. Puntúa las 5 amenazas más relevantes con DREAD (o CVSS si prefieres algo más objetivo) y ordénalas. 6. **Deriva contramedidas**. Para cada amenaza top, escribe un requisito verificable (p. ej. "todo flujo de login usa TLS 1.2+ y rate-limiting de 5 intentos/min"). 7. **Versiona el modelo**. Guarda `tm.py` y `dfd.png` en el repo junto al código; el modelo evoluciona con el sistema.

Nota ética: el modelado de amenazas es una actividad defensiva de diseño. No requiere atacar sistemas; se practica sobre tus propios diseños.

Ejercicios

1. Dibuja el DFD de un formulario de contacto con almacenamiento en base de datos.
2. Aplica STRIDE al flujo "API → servicio de pagos externo" y lista 6 amenazas.
3. Puntúa dos amenazas con DREAD y argumenta por qué el scoring es discutible.
4. Convierte tres amenazas en requisitos de seguridad verificables.
5. Modela con pytm un sistema con dos trust boundaries y genera el informe.
6. Propón una alternativa a DREAD y justifica cuándo la usarías.

Reto verificable

Entrega un modelo de amenazas completo de un sistema con al menos dos trust boundaries.

Criterio de aceptación: incluye (a) un DFD con procesos, almacenes, flujos y límites de confianza; (b) una tabla STRIDE con mínimo 8 amenazas categorizadas; (c) priorización justificada de al menos 5; y (d) una contramedida/requisito verificable por cada amenaza priorizada. El modelo debe ser versionable (Threat Dragon JSON o pytm).

⚠ Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
El DFD tiene 200 elementos y nadie lo entiende	Nivel de detalle excesivo. Modela al nivel de arquitectura, no de función.
Se listan amenazas pero no mitigaciones	El modelo no aporta valor sin contramedidas accionables. Cierra el ciclo.
DREAD da puntuaciones incoherentes entre personas	Es subjetivo por diseño. Calibra con guías o cambia a CVSS/ábaco de riesgo.
El modelo se hace una vez y se olvida	Debe vivir con el código. Revisalo en cada cambio de arquitectura.
Se ignora la categoría Repudiation	Falta pensar en logging/no repudio. Añade trazabilidad como requisito.

? Preguntas frecuentes

? **¿Cuándo hago threat modeling: al inicio o continuamente?** Al inicio del diseño y luego incrementalmente en cada cambio arquitectónico significativo. Es un documento vivo, no un entregable único.

? **¿STRIDE cubre todas las amenazas posibles?** No, pero da una cobertura sistemática de las categorías más comunes. Complementa con árboles de ataque o MITRE ATT&CK para escenarios específicos.

? **¿Vale la pena DREAD si Microsoft lo abandonó?** Como introducción pedagógica sí; en producción muchos equipos prefieren CVSS o una matriz probabilidad×impacto por su mayor consistencia.

? **¿Puedo automatizar el modelado de amenazas?** Parcialmente. Herramientas como pytm generan amenazas candidatas a partir del modelo, pero el juicio experto sobre contexto y priorización sigue siendo humano.

🔗 Referencias

- Adam Shostack, *Threat Modeling: Designing for Security*, Wiley 2014.
- OWASP Threat Modeling Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html
- OWASP Threat Dragon — <https://owasp.org/www-project-threat-dragon/>
- pytm — <https://github.com/OWASP/pytm>
- Microsoft STRIDE — <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats>

➡ Siguiente clase

Clase 238 - SAST: analisis estatico de codigo